

Company Website

Shell

Windows

```
$ cd onedrive\desktop\code  
$ mkdir company  
$ cd company  
$ python -m venv .venv  
$ .venv\Scripts\Activate.ps1  
(.venv) $
```

macOS

```
$ cd ~/desktop/code  
$ mkdir company  
$ cd company  
$ python3 -m venv .venv  
$ source .venv/bin/activate  
(.venv) $
```

Shell

```
(.venv) $ python -m pip install django~=5.0.0  
(.venv) $ python -m pip install black  
(.venv) $ django-admin startproject django_project .  
(.venv) $ python manage.py startapp pages
```

Code

```
# django_project/settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "pages", # new
]
```

Initialize the database with `migrate` and start the local web server with `runserver`.

Shell

```
(.venv) $ python manage.py migrate
(.venv) $ python manage.py runserver
```

Then navigate to `http://127.0.0.1:8000/` to see the Django welcome page.

Project-Level Templates

We previously saw that Django expects template files to be located within an app in a directory called `templates` and that it was a best practice to namespace this further by adding the directory name again. In other words, template files for a `pages` app would be located in a directory at `pages/templates/pages/`.

However, many Django developers favor another approach: creating a single project-level `templates` directory and placing *all* templates within it. This makes finding and updating all the templates in one location easier. By tweaking our `django_project/settings.py` file, we can also tell Django to look in this directory for templates.

Shell

```
(.venv) $ mkdir templates
```

Next, we need to update `django_project/settings.py` to tell Django where our new templates directory is. This requires a one-line change to the "DIRS" configuration under `TEMPLATES`.

Code

```
# django_project/settings.py
TEMPLATES = [
    {
        ...
        "DIRS": [BASE_DIR / "templates"], # new
        ...
    },
]
```

We'll use that approach to organizing templates for the rest of the book. Create a new file called `home.html` within the `templates` directory. You can do this within your text editor: in Visual Studio Code, go to the top left of your screen, click "File," and then "New File." Make sure to name and save the file in the correct location.

For now, the `home.html` file will have a simple headline.

Code

```
<!-- templates/home.html -->  
<h1>Company Homepage</h1>
```

Code

```
# pages/views.py
from django.shortcuts import render

def home_page_view(request): # new
    return render(request, "home.html")
```

Next are the views, both the project-level `urls.py` file, which serves as our website's entry point, and the app-specific `urls.py` file, which has the specific route and view for the home page.

The `django_project/urls.py` file is the initial entry point for all URL requests to our project. We must import the `include` function at the top and then use it to include the URL routes from the `pages` application, which will be set to the empty string of `""`.

Code

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include # new

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("pages.urls")), # new
]
```

Code

```
# pages/urls.py
from django.urls import path

from .views import home_page_view

urlpatterns = [
    path("", home_page_view),
]
```

Start up the development web server using the `runserver` command.

Shell

```
(.venv) $ python manage.py runserver
```

If you navigate `http://127.0.0.1:8000/`, the home page is now visible.



Company Homepage

FBV Company Homepage

Easy enough, right? So far, the only new concept introduced is the project-level `templates` directory.

Code

```
# pages/views.py
from django.shortcuts import render

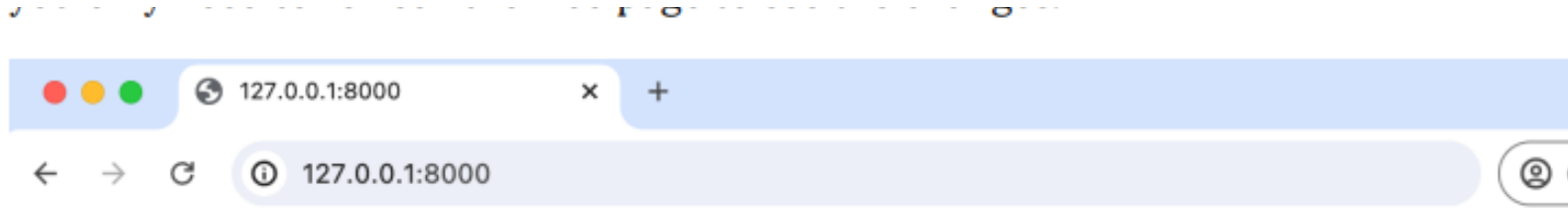
def home_page_view(request):
    context = { # new
        "inventory_list": ["Widget 1", "Widget 2", "Widget 3"],
        "greeting": "THAnk you FOR visitING.",
    }
    return render(request, "home.html", context)
```

Code

```
<!-- templates/home.html -->
<h1>Company Homepage</h1>
<p>The current date and time is: {% now "DATETIME_FORMAT" %}</p>
<p>There are {{ inventory_list|length }} items of inventory.
<ul>
    {% for item in inventory_list %}
    <li>{{ item }}</li>
    {% endfor %}
</ul>
<p>{{ greeting| title }}</p>
{% comment %}Add more content here!{% endcomment %}
```

Update the `home.html` template file with the following code. The `now`⁶² tag displays the current date and/or time using `DATE_FORMAT`⁶³, one of several display options. Next, display the number of items in the `inventory_list` using the `length`⁶⁴ filter, which works on both strings and lists.

Then, loop over each item in the `for`⁶⁵ tag. The general syntax is `{% for item in item_list %}` where `item` is a variable name representing the current item in the loop and `item_list` is the sequence we are looping over. It is also critical to include an `{% endfor %}` tag to conclude any `for` loop. In this case, the sequence is named `inventory_list`. We could name the variable anything we like, but a description such as `item` is a common choice that makes the code easier to reason about. Finally, we use the `title`⁶⁶ filter to convert a string into titlecase where each word starts with an uppercase letter followed by lowercase letters.



Company Homepage

The current date and time is: June 30, 2024, 8:25 p.m.

There are 3 items of inventory.

- Widget 1
- Widget 2
- Widget 3

Thank You For Visiting.

Company Homepage with Context

Django has function-based views and class-based views.

The problem with function-based views was that there was no easy way to extend or customize them. As projects grow, this becomes more and more of an issue.

Classes are a fundamental part of Python that rely on object oriented programming (OOP) and inheritance, so one class can inherit attributes and methods from another. That means we don't have to include all the logic for a view in one place but rather can abstract common patterns and then customize or extend them as needed.

Once you have used **generic class-based views** for a while, they become elegant and efficient ways to write code. You can often modify a single method on one to do custom behavior rather than rewriting everything from scratch, which makes it easier to understand someone else's code.

However, this comes at the cost of complexity and requires a leap of faith because it takes a **long time to understand how they work under the hood**.

Template View

Create a second web page for our Company site, this time using **TemplateView**, a generic class-based view.

Code

```
# pages/views.py
from django.shortcuts import render
from django.views.generic import TemplateView # new

def home_page_view(request):
    context = {
        "inventory_list": ["Widget 1", "Widget 2", "Widget 3"],
        "greeting": "THAnk you FOR visitING.",
    }
    return render(request, "home.html", context)

class AboutPageView(TemplateView): # new
    template_name = "about.html"
```

Code

```
# pages/urls.py
from django.urls import path

from .views import home_page_view, AboutPageView # new

urlpatterns = [
    path("about/", AboutPageView.as_view()), # new
    path("", home_page_view),
]
```

Note the addition of the `as_view()` method that returns a callable view.

Code

```
<!-- templates/about.html -->  
<h1>Company About Page</h1>
```

get_context_data()

One of the most powerful, useful, and commonly used methods in Django is `get_context_data()`⁷³. It is the recommended approach for updating the template context in a generic class-based view. Let's use it now to add context data to the About page.

Code

```
# pages/views.py
...
class AboutPageView(TemplateView):
    template_name = "about.html"

    def get_context_data(self, **kwargs): # new
        context = super().get_context_data(**kwargs)
        context["contact_address"] = "123 Main Street"
        context["phone_number"] = "555-555-5555"
        return context
...
```

Code

```
<!-- templates/about.html -->
<h1>Company About Page</h1>
<p>The company address is {{ contact_address }} and the phone number is
  {{ phone_number }}.</p>
```

Refresh the About page in your web browser to see this information displayed.



Company About Page

The company address is 123 Main Street and the phone number is 555-555-5555.

Company About Page with Context

Within the `templates` directory, create a `base.html` file containing a header with links to our home page and about pages. This is our parent template, which all other child templates will inherit from. To define what areas can be overridden, we'll use `block`⁷⁴ tags in the syntax `{% block content %}` and `{% endblock %}`. Anything within the block tags can be overridden in a child template.

Code

```
<!-- templates/base.html -->
<header>
  <a href="/">Home</a> |
  <a href="/about">About</a>
</header>

{% block content %}{% endblock %}
```

The extends tag allows us to establish parent/child relationships between our templates by specifying the parent template.

Add it to the top of the home.html and about.html templates. Then define our child template content with {% block content %} and {% endblock %} tags.

Code

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block content %}
<h1>Company Homepage</h1>
<p>The current date and time is: {% now "DATETIME_FORMAT" %}</p>
<p>There are {{ inventory_list|length }} items of inventory.
<ul>
  {% for item in inventory_list %}
    <li>{{ item }}</li>
  {% endfor %}
</ul>
<p>{{ greeting| title }}</p>
{% comment %}Add more content here!{% endcomment %}
{% endblock %}
```

Code

```
<!-- templates/about.html -->
{% extends "base.html" %}
{% block content %}
<h1>Company About Page</h1>
<p>The company address is {{ contact_address }} and the phone number is
  {{ phone_number }}.</p>
{% endblock %}
```



[Home](#) | [About](#)

Company Homepage

The current date and time is: June 30, 2024, 8:30 p.m.

There are 3 items of inventory.

- Widget 1
- Widget 2
- Widget 3

Thank You For Visiting.

Company Home Page with Base Template



[Home](#) | [About](#)

Company About Page

The company address is 123 Main Street and the phone number is 555-555-5555.

Company About Page with Base Template

Code

```
# pages/urls.py
from django.urls import path

from .views import home_page_view, AboutPageView

urlpatterns = [
    path("about/", AboutPageView.as_view(), name="about"), # new
    path("", home_page_view, name="home"), # new
]
```

Now, whenever we want to refer to a specific URL path, we can do so using the named URL in our template via the built-in [url](#)⁷⁸ template tag. Update the `base.html` file with the code below.

Code

```
<!-- templates/base.html -->
<header>
  <a href="{% url 'home' %}">Home</a> |
  <a href="{% url 'about' %}">About</a>
</header>

{% block content %}{% endblock %}
```

Code

```
# pages/tests.py
from django.test import SimpleTestCase

class HomepageTests(SimpleTestCase):
    def test_url_exists_at_correct_location(self):
        response = self.client.get("/")
        self.assertEqual(response.status_code, 200)

class AboutpageTests(SimpleTestCase):
    def test_url_exists_at_correct_location(self):
        response = self.client.get("/about/")
        self.assertEqual(response.status_code, 200)
```

Shell

```
(.venv) $ python manage.py test  
Found 2 test(s).  
System check identified no issues (0 silenced).  
..
```

```
-----  
Ran 2 tests in 0.003s
```

OK

Code

```
# pages/tests.py
from django.test import SimpleTestCase
from django.urls import reverse  # new

class HomepageTests(SimpleTestCase):
    def test_url_exists_at_correct_location(self):
        response = self.client.get("/")
        self.assertEqual(response.status_code, 200)

    def test_url_available_by_name(self):  # new
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)

class AboutpageTests(SimpleTestCase):
    def test_url_exists_at_correct_location(self):
        response = self.client.get("/about/")
        self.assertEqual(response.status_code, 200)

    def test_url_available_by_name(self):  # new
        response = self.client.get(reverse("about"))
        self.assertEqual(response.status_code, 200)
```

Shell

```
(.venv) $ python manage.py test
Found 4 test(s).
System check identified no issues (0 silenced).
..
-----
Ran 4 tests in 0.005s
```

OK

Code

```
# pages/tests.py
from django.test import SimpleTestCase
from django.urls import reverse

class HomepageTests(SimpleTestCase):
    def test_url_exists_at_correct_location(self):
        response = self.client.get("/")
        self.assertEqual(response.status_code, 200)

    def test_url_available_by_name(self):
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)

    def test_template_name_correct(self): # new
        response = self.client.get(reverse("home"))
        self.assertTemplateUsed(response, "home.html")

    def test_template_content(self): # new
        response = self.client.get(reverse("home"))
        self.assertContains(response, "<h1>Company Homepage</h1>")
```

```
class AboutpageTests(SimpleTestCase):
    def test_url_exists_at_correct_location(self):
        response = self.client.get("/about/")
        self.assertEqual(response.status_code, 200)

    def test_url_available_by_name(self):
        response = self.client.get(reverse("about"))
        self.assertEqual(response.status_code, 200)

    def test_template_name_correct(self): # new
        response = self.client.get(reverse("about"))
        self.assertTemplateUsed(response, "about.html")

    def test_template_content(self): # new
        response = self.client.get(reverse("about"))
        self.assertContains(response, "<h1>Company About Page</h1>")
```

Run the tests one last time to check our new work. Everything should pass.

Shell

```
(.venv) $ python manage.py test
Found 8 test(s).
System check identified no issues (0 silenced).
..
-----
Ran 8 tests in 0.006s
```

OK

Shell

```
(.venv) $ git init  
(.venv) $ git status
```

Then, create a `.gitignore` file instructing Git on what not to track. We will focus on three areas: the `.venv` directory with virtual environments, the `__pycache__` directory with compiled bytecode, and the database file, `db.sqlite3`.

`.gitignore`

```
.venv/  
__pycache__/  
db.sqlite3
```

The next step is to create a `requirements.txt` file listing the contents of our virtual environment.

Shell

```
(.venv) $ pip freeze > requirements.txt  
(.venv) $ git status
```

Shell

```
(.venv) $ git status  
(.venv) $ git add -A  
(.venv) $ git commit -m "initial commit"
```

Over on GitHub [create a new repo](#)⁸³ called `company-website` and make sure to select the “Private” radio button. Then click on the “Create repository” button.

On the next page, scroll down to where it says “`or push an existing repository from the command line`.” Copy and paste the two commands there into your terminal.

It should look like the below, albeit instead of `wsvincent` as the username, it will be your GitHub username.

Shell

```
(.venv) $ git remote add origin https://github.com/wsvincent/company-website.git  
(.venv) $ git branch -M main  
(.venv) $ git push -u origin main
```
